

Task 4 项目总结报告

从数据到数据产品——B站热门视频交互式看板部署

姓名：钱欣羽 • 提交日期：2026-07-25

一、问题定义与目标

前三个任务完成了“从数据到洞察”的分析链路（Task 1 抓取、Task 2 清洗、Task 3 探索性分析）。Task 4 的任务是把分析成果转化为一个可交付的数据产品：一个任何人都可以在线访问的交互式 HTML 数据看板，并完成从本地代码到 GitHub Pages 上线的工程化部署。核心目标是实现从“数据消费者”到“数据产品创造者”的身份跃迁，获得一份可写进简历的完整作品。

交付物包含四部分：① 单个自包含 HTML 看板文件（CSS/JS 全内联，可直接打开）；② GitHub Pages 在线访问链接；③ 本 PDF 项目总结报告；④ Git 操作过程记录（SSH 配置、git log 等）。

二、看板规划与信息架构

看板采用单页信息架构，自上而下分为五个模块：标题区（项目名与作者）、核心指标卡片区（6 张关键数字卡）、交互图表区（6 张 Plotly 图表）、关键洞察区（6 条结论 + 4 个可验证假设）、页脚部署说明。布局使用 CSS Grid/Flex 实现响应式，在桌面与手机宽度下均保持可读。

指标卡选取逻辑：样本量(120)与字段数(18)交代数据规模；平均/中位/头部播放量与平均点赞，直观呈现“长尾”特征，为后续图表埋下伏笔。

三、技术实现方案

使用 Plotly.js 作为交互图表库。为满足“单个 HTML 文件、所有 CSS/JS 内嵌、任意浏览器直接打开”的要求，将完整的 Plotly.js（约 4.5 MB）直接内联进 HTML 的 `<script>` 标签，使文件离线可用，同时天然适配 GitHub Pages 的在线访问。

图表由 Python (pandas + plotly) 在本地生成数据并序列化为 Plotly 的 data/layout JSON，再在页面中通过 Plotly.newPlot 渲染。交互功能覆盖：悬停显示精确数值、框选缩放、图例点击筛选分区、以及“按分区筛选”下拉框。看板数据已内联，不依赖外部接口。

四、核心图表与数据洞察

以下 6 张图覆盖单变量分布、分组对比、双变量关联与高级可视化，每张均配编号、标题与文字解读。

图1 播放量分布直方图（log10 分箱）

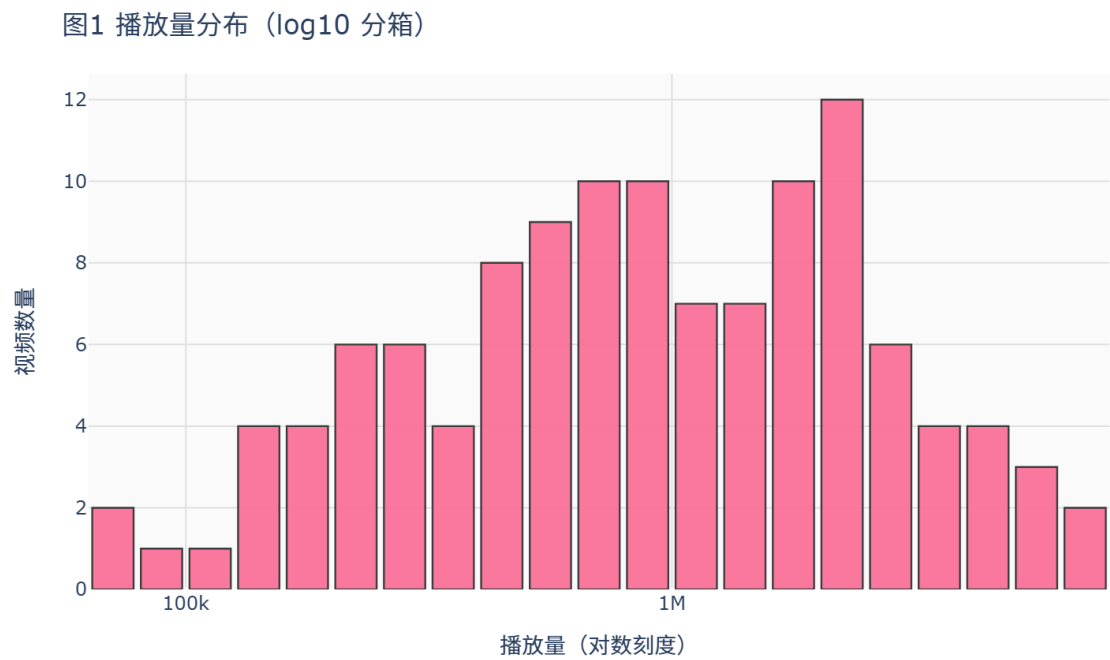


图1 播放量分布直方图（log10 分箱）

解读：多数视频集中在较低播放量区间，少数头部视频极高，呈典型右偏长尾。横轴采用 $\log_{10}$ (播放量) 对数刻度，能更清楚地看清长尾结构。

图2 六大分区播放量分布对比

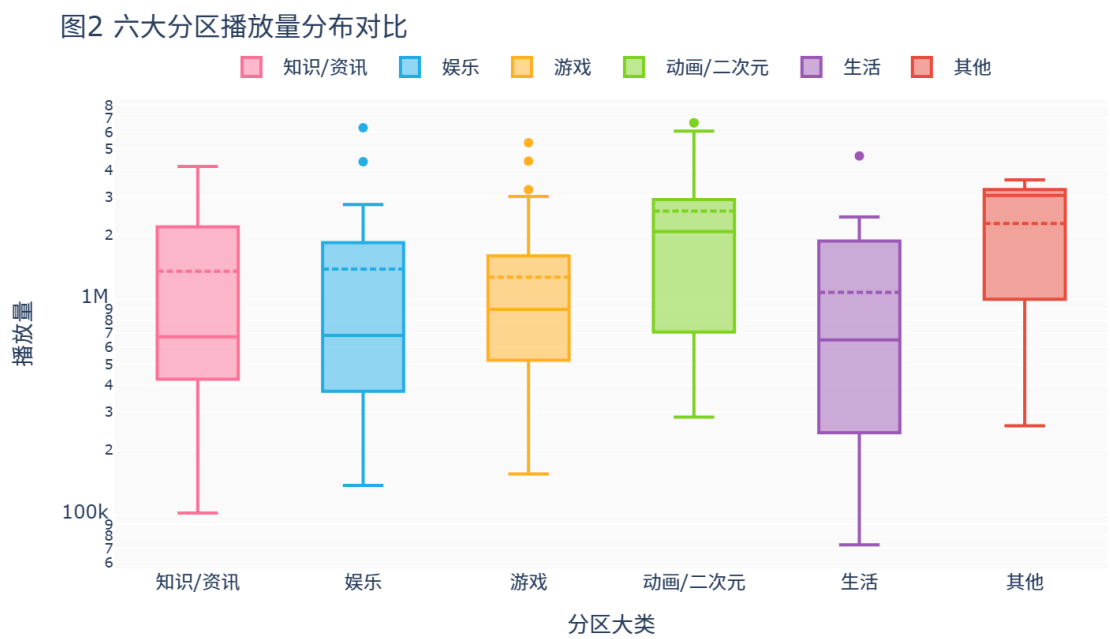


图2 六大分区播放量分布对比

解读：各分区播放量中位数与离散程度存在明显差异，动画/二次元及“其他”分区相对较高；但“其他”仅有 5 条样本，不能作为稳健结论。各分区均存在上方离群点（爆款）。看板中点击图上方图例可隐藏/显示某一分区。

图3 核心变量相关系数热力图

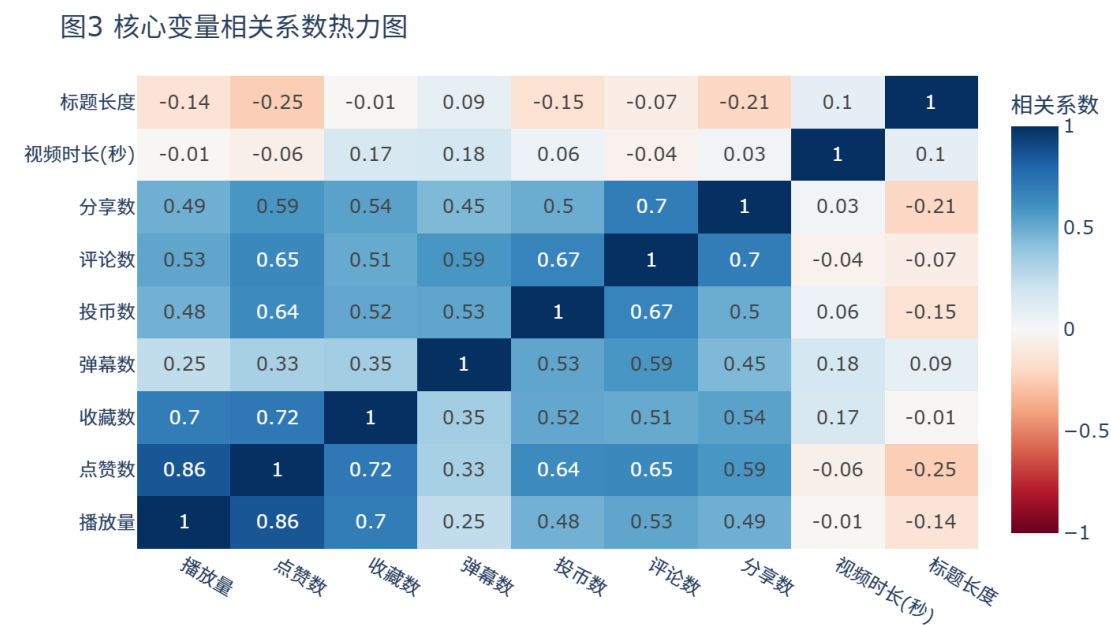


图3 核心变量相关系数热力图

解读：播放量与点赞数($r=0.86$)、收藏数($r=0.70$)强正相关；视频时长与播放量几乎无关($r\approx-0.01$)。解释播放量差异最应关注点赞与收藏。

图4 播放量 vs 点赞数（双对数 + 回归线）

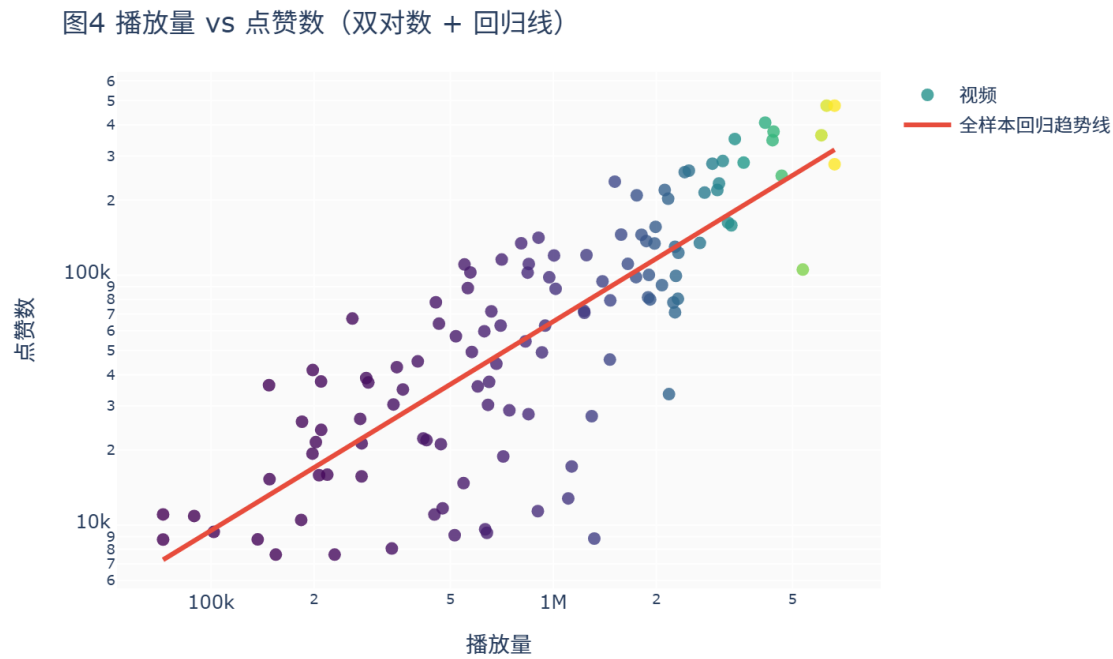


图4 播放量 vs 点赞数（双对数 + 回归线）

解读：双对数下散点沿红色回归线分布，二者共变关系稳健。二者为同期累计数据，只能说明共变，不能推断因果。看板中可通过下拉框筛选不同分区的视频散点，红色趋势线代表全样本整体回归关系。

图5 不同发布时段播放量中位数

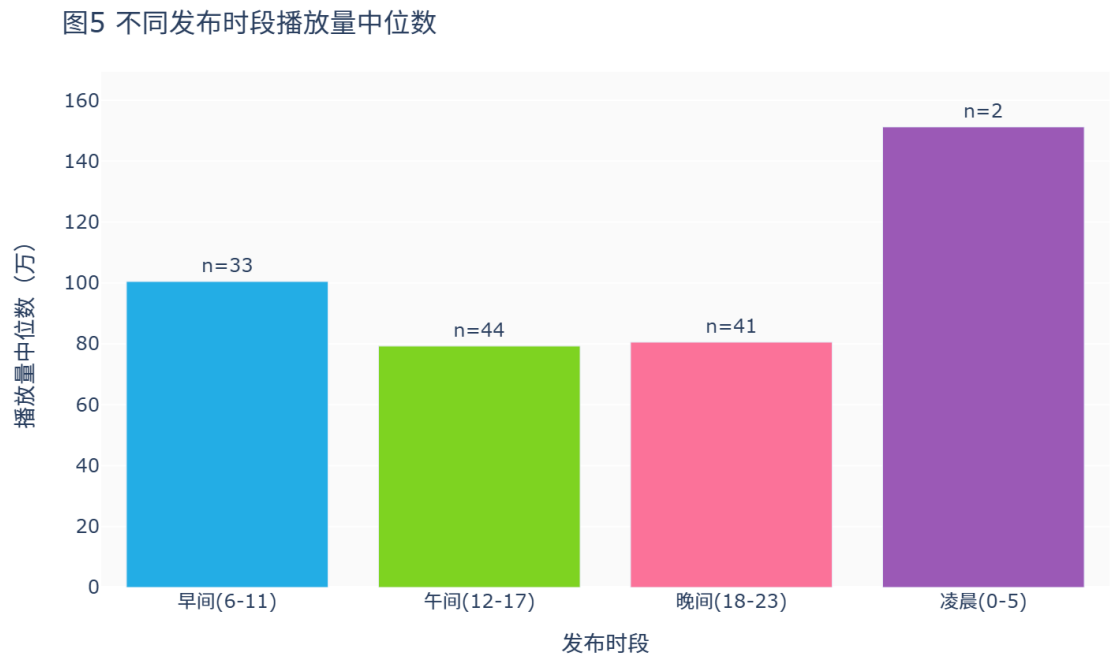


图5 不同发布时段播放量中位数

解读：早/中/晚中位数接近（约 80 - 100 万），差异有限；凌晨柱虽高但样本量 n=2，结论不可靠，已用柱顶标注提示。

图6 标题长度与平均播放量关系

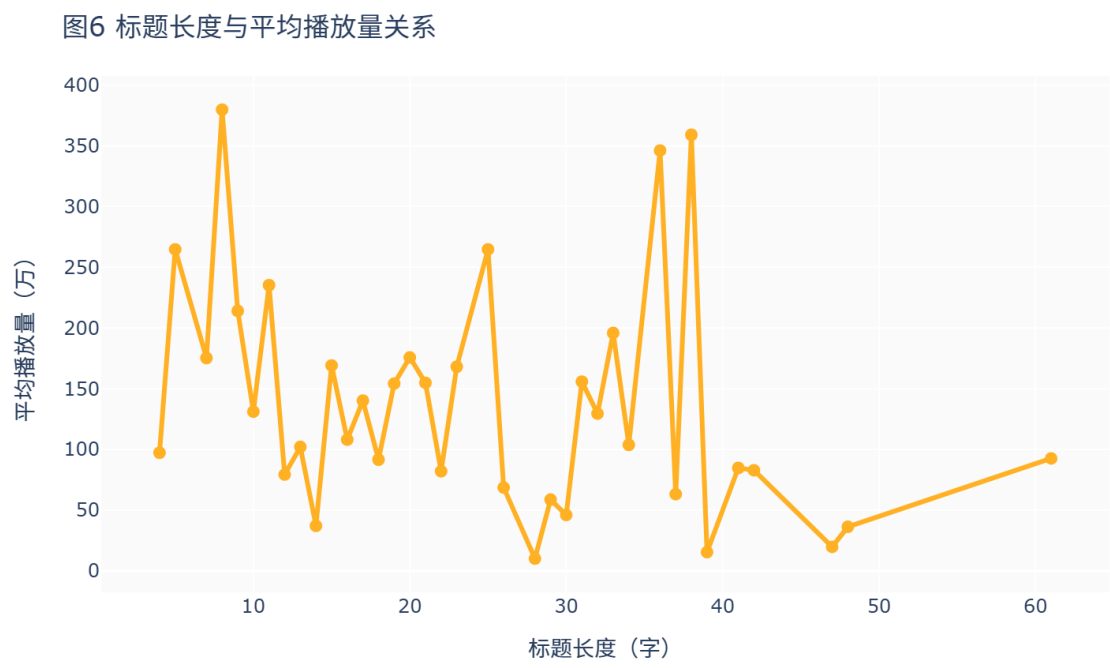


图6 标题长度与平均播放量关系

解读：各标题长度区间平均播放量波动较大，无单调趋势，说明靠“堆字数”并不能稳定提升热度。

五、技术踩坑记录

1. 中文乱码与混排间隔：图表中文显示为方块，PDF 宋体正文在两端对齐时中英数混排间隔被拉宽。解决——图表用 SimHei；PDF 保持宋体正文，启用 CJK 逐字换行并收紧空格伸缩参数，混排即恢复紧凑。
2. 单文件离线化：Plotly 默认引用 CDN，断网打开空白。解决——将完整 `plotly.min.js` 内联进 HTML，文件虽增至约 4.4 MB，但真正实现“任意浏览器直接打开”。
3. 样本量陷阱：周末仅 4 条、凌晨仅 2 条、分区“其他”仅 5 条。若直接对这些分组下结论会误导，故在图表与报告中显式标注样本量，并声明“仅作参考示意”。
4. 相关性 $\neq$ 因果：播放量与点赞数强相关，但二者是同期累计数据。初稿曾写成“点赞带动播放”，经审查改为“共变关系”，避免因果误判。
5. Git 身份缺失：首次提交因未配置 `user.name/email` 报错。解决——在仓库内执行 `git config user.name/user.email`。注意邮箱应与 GitHub 账户一致，否则贡献不计入主页。
6. SSH 公钥未添加：生成密钥后推送被拒。需把 `id_ed25519.pub` 内容手动粘贴到 GitHub 的 Settings→SSH keys，且仓库需先在网页端创建。
7. Pages 分支选择：GitHub Pages 需在 Settings→Pages 手动选择部署分支（main）与根目录，保存后等待 2-5 分钟才生效，并非立即可访问。

六、智能体使用心得

本任务中，我使用智能体辅助完成看板信息架构梳理、Plotly 图表代码生成、HTML 与 CSS 响应式布局优化、Git 与 SSH 命令排查，以及 PDF 报告格式校对。智能体提高了代码编写和问题定位效率，但生成结果仍需要人工核验。我通过本地打开 HTML、测试悬停、缩放和筛选功能，核对统计数值，并检查 Git 推送结果，发现并修正了对数直方图显示、回归线筛选和字体兼容等问题。由此认识到，智能体适合作为辅助工具，不能替代数据判断、功能测试和最终责任。

本次 Task 4 综合运用了 Python 数据分析、Plotly 可视化与 Git/GitHub Pages 部署等工具。通过先在小数据上验证图表效果，再整合到完整看板中的方式，逐步掌握了从本地脚本到在线产品的发布链路。

体会：工具可以大幅提升效率，但每一步仍需自己理解其原理与输出含义。例如，Git 的提交、分支、远程仓库等概念只有在亲自配置并看到推送结果后才会真正内化；Plotly 的交互参数也需要对照文档反复调试才能达到预期效果。

七、学习反思与收获

技能层面：掌握了 Plotly 交互可视化的基本范式、单文件前端的离线打包思路，以及 Git/SSH/GitHub Pages 的完整发布链路。认知层面：更深刻理解了“分析洞察”与“产品交付”之间的差距——看板不仅要图表正确，还要信息层次清晰、交互顺手、在任何屏幕都可读。

不足与改进：当前看板数据为静态快照，未来可接入实时数据源；样本量局限表明若要做稳健结论，需要更完整、更大规模的采集。本次也暴露出对“统计显著性”意识的欠缺，后续应补强假设检验知识。

八、部署与提交清单

本地已完成：生成 `index.html`（自包含看板）、导出图表、初始化 Git 仓库并提交（分支 `main`，提交记录见 Git 操作过程记录）。已在 GitHub 网页端手动完成：① 新建仓库；② 在 Settings→SSH keys 添加本机公钥；③ 执行 `git remote add / git push`；④ 在

Settings→Pages 启用 main 分支部署；⑤ 访问 <https://24212486-bit.github.io/bilibili-edashboard/> 验证。

提交材料：本 PDF 报告、index.html 看板文件、GitHub Pages 链接、以及 Git 操作过程记录（含 SSH 公钥指纹、git log 截图/文本）。